

ASSESSMENT TOOL 2 - SCHEMA ANALYSIS

Course Code: DSA0504	Course Title: Query Processing for Data Science
CO Assessed: CO2	Assessment: Tool 2 – Schema Analysis
Weightage: 40	Total Marks: 25 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)	
Student Name: J.Subhaashree	Reg. No.: 192424008
Section / Batch: 2024	Date of Submission: 3/8/26
Faculty In-charge: Dr. R.Mahesh Muthulakshmi	Project Mode: Individual Submission

1) University Course Registration System

1. Aim

To analyze and implement a database schema using MySQL by creating tables, defining primary keys and foreign keys, and establishing relationships.

2. Schema Design

Student

- StudentID (PK)
- StudentName
- Department

Course

- CourseID (PK)
- CourseName
- Credits

Enrollment

- EnrollmentID (PK)
- StudentID (FK)
- CourseID (FK)
- Semester

3. SQL Code

-- Create database

```
CREATE DATABASE college;
```

-- Use database

```
USE college;
```

-- Create Student table

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(50),  
    Department VARCHAR(50)  
);
```

-- Create Course table

```
CREATE TABLE Course (  
    CourseID INT PRIMARY KEY,
```

```
    CourseName VARCHAR(50),  
    Credits INT  
);
```

-- Create Enrollment table

```
CREATE TABLE Enrollment (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT,  
    CourseID INT,  
    Semester VARCHAR(20),  
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),  
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)  
);
```

-- Display tables

```
SHOW TABLES;
```

-- Display table structures

```
DESCRIBE Student;
```

```
DESCRIBE Course;
```

```
DESCRIBE Enrollment;
```

-- Insert sample data into Student

```
INSERT INTO Student VALUES
```

```
(1, 'Ram', 'CSE'),
```

```
(2, 'Priya', 'AI');
```

-- Insert sample data into Course

```
INSERT INTO Course VALUES
```

```
(101, 'DBMS', 4),
```

```
(102, 'Python', 3);
```

-- Insert sample data into Enrollment

```
INSERT INTO Enrollment VALUES
```

```
(1, 1, 101, 'Semester 1'),
```

```
(2, 2, 102, 'Semester 1');
```

-- View data

```
SELECT * FROM Student;
```

```
SELECT * FROM Course;
```

```
SELECT * FROM Enrollment;
```

-- Schema analysis using JOIN

```
SELECT
```

```
Student.StudentName,  
Course.CourseName,  
Enrollment.Semester  
FROM Enrollment  
JOIN Student  
ON Enrollment.StudentID = Student.StudentID  
JOIN Course  
ON Enrollment.CourseID = Course.CourseID;
```

4. Output

```
0 rows in set (0.0020 sec)  
MySQL localhost:33060+ ssl SQL > CREATE DATABASE college;  
Query OK, 1 row affected (0.0196 sec)  
MySQL localhost:33060+ ssl SQL > USE college;  
Default schema set to 'college'.  
Fetching global names, object names from 'college' for auto-completion... Press ^C to stop.  
MySQL localhost:33060+ ssl college SQL > CREATE TABLE Student (  
-> StudentID INT PRIMARY KEY,  
-> StudentName VARCHAR(50),  
-> Department VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.0612 sec)  
MySQL localhost:33060+ ssl college SQL > CREATE TABLE Course (  
-> CourseID INT PRIMARY KEY,  
-> CourseName VARCHAR(50),  
-> Credits INT  
-> );  
Query OK, 0 rows affected (0.0545 sec)  
MySQL localhost:33060+ ssl college SQL > CREATE TABLE Enrollment (  
-> EnrollmentID INT PRIMARY KEY,  
-> StudentID INT,  
-> CourseID INT,  
-> Semester VARCHAR(20),  
-> FOREIGN KEY (StudentID) REFERENCES Student(StudentID),  
-> FOREIGN KEY (CourseID) REFERENCES Course(CourseID)  
-> );  
Query OK, 0 rows affected (0.0776 sec)  
MySQL localhost:33060+ ssl college SQL > SHOW TABLES;  
+-----+  
| Tables_in_college |  
+-----+  
| course             |  
| enrollment         |  
| student            |  
+-----+
```

3 rows in set (0.0035 sec)

MySQL localhost:33060+ ssl college SQL > DESCRIBE Student;

Field	Type	Null	Key	Default	Extra
StudentID	int	NO	PRI	NULL	
StudentName	varchar(50)	YES		NULL	
Department	varchar(50)	YES		NULL	

3 rows in set (0.0110 sec)

MySQL localhost:33060+ ssl college SQL > DESCRIBE Course;

Field	Type	Null	Key	Default	Extra
CourseID	int	NO	PRI	NULL	
CourseName	varchar(50)	YES		NULL	
Credits	int	YES		NULL	

3 rows in set (0.0047 sec)

MySQL localhost:33060+ ssl college SQL > DESCRIBE Enrollment;

Field	Type	Null	Key	Default	Extra
EnrollmentID	int	NO	PRI	NULL	
StudentID	int	YES	MUL	NULL	
CourseID	int	YES	MUL	NULL	
Semester	varchar(20)	YES		NULL	

4 rows in set (0.0043 sec)

MySQL localhost:33060+ ssl college SQL > INSERT INTO Student VALUES
-> (1, 'Ram', 'CSE'),
-> (2, 'Priya', 'AI');

Query OK, 2 rows affected (0.0625 sec)

Records: 2 Duplicates: 0 Warnings: 0

MySQL localhost:33060+ ssl college SQL > INSERT INTO Course VALUES
-> (101, 'DBMS', 4),
-> (102, 'Python', 3);

Query OK, 2 rows affected (0.0159 sec)

Records: 2 Duplicates: 0 Warnings: 0

MySQL localhost:33060+ ssl college SQL > INSERT INTO Enrollment VALUES
-> (1, 1, 101, 'Semester 1'),
-> (2, 2, 102, 'Semester 1');

Query OK, 2 rows affected (0.0354 sec)

```

Records: 2  Duplicates: 0  Warnings: 0
MySQL localhost:33060+ ssl college SQL> SELECT * FROM Student;
+-----+-----+-----+
| StudentID | StudentName | Department |
+-----+-----+-----+
| 1 | Ram | CSE |
| 2 | Priya | AI |
+-----+-----+-----+
2 rows in set (0.0009 sec)

MySQL localhost:33060+ ssl college SQL> SELECT * FROM Course;
+-----+-----+-----+
| CourseID | CourseName | Credits |
+-----+-----+-----+
| 101 | DBMS | 4 |
| 102 | Python | 3 |
+-----+-----+-----+
2 rows in set (0.0011 sec)

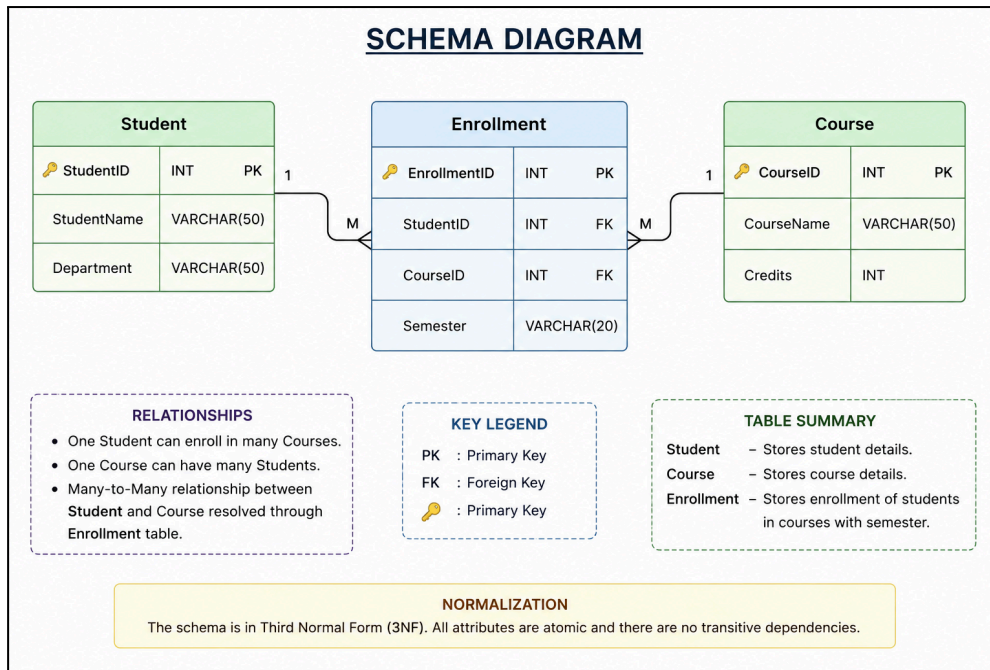
MySQL localhost:33060+ ssl college SQL> SELECT * FROM Enrollment;
+-----+-----+-----+-----+
| EnrollmentID | StudentID | CourseID | Semester |
+-----+-----+-----+-----+
| 1 | 1 | 101 | Semester 1 |
| 2 | 2 | 102 | Semester 1 |
+-----+-----+-----+-----+
2 rows in set (0.0012 sec)

MySQL localhost:33060+ ssl college SQL> SELECT Student.StudentName,
-> Course.CourseName,
-> Enrollment.Semester
-> FROM Enrollment
-> JOIN Student
-> ON Enrollment.StudentID = Student.StudentID
-> JOIN Course
-> ON Enrollment.CourseID = Course.CourseID;
+-----+-----+-----+
| StudentName | CourseName | Semester |
+-----+-----+-----+
| Ram | DBMS | Semester 1 |
| Priya | Python | Semester 1 |
+-----+-----+-----+

```

5. Schema Analysis

- Primary keys: StudentID, CourseID, EnrollmentID
- Foreign keys: StudentID, CourseID
- Relationship: Many-to-many relationship between Student and Course through Enrollment.
- Normalization: The schema satisfies 3NF.



6. Conclusion

The schema was successfully created and analyzed using MySQL. The relationships between the tables were established using primary and foreign keys, ensuring data integrity and reducing redundancy.

2) Hospital Management System

1) Aim

To create and analyze the Hospital Management System schema using MySQL by defining tables, primary keys, foreign keys, and relationships.

2) Schema Design

Patient Table

- PatientID (PK)
- Name
- Age

Doctor Table

- DoctorID (PK)
- DoctorName
- Specialization

Appointment Table

- AppointmentID (PK)
- PatientID (FK)
- DoctorID (FK)
- Date

Prescription Table

- PrescriptionID (PK)
- AppointmentID (FK)
- Medicine

Relationship Analysis

- One patient can have many appointments (**1:M**).
- One doctor can have many appointments (**1:M**).
- One appointment can have many prescriptions (**1:M**).

3) MySQL Code

-- Create database

```
CREATE DATABASE hospital;
```

-- Use database

```
USE hospital;
```

-- Create Patient table

```
CREATE TABLE Patient (  
    PatientID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT  
);
```

-- Create Doctor table

```
CREATE TABLE Doctor (  
    DoctorID INT PRIMARY KEY,  
    DoctorName VARCHAR(50),  
    Specialization VARCHAR(50)  
);
```

-- Create Appointment table

```
CREATE TABLE Appointment (  
    AppointmentID INT PRIMARY KEY,  
    PatientID INT,  
    DoctorID INT,  
    Date DATE,  
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),  
    FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID)  
);
```

-- Create Prescription table

```
CREATE TABLE Prescription (  
    PrescriptionID INT PRIMARY KEY,  
    AppointmentID INT,  
    Medicine VARCHAR(50),  
    FOREIGN KEY (AppointmentID) REFERENCES Appointment(AppointmentID)
```

```
);
```

```
-- Show tables
```

```
SHOW TABLES;
```

```
-- Display structure
```

```
DESCRIBE Patient;  
DESCRIBE Doctor;  
DESCRIBE Appointment;  
DESCRIBE Prescription;
```

```
-- Insert sample values
```

```
INSERT INTO Patient VALUES  
(1, 'Ram', 25),  
(2, 'Priya', 30);
```

```
INSERT INTO Doctor VALUES  
(101, 'Dr. Kumar', 'Cardiology'),  
(102, 'Dr. Meena', 'Neurology');
```

```
INSERT INTO Appointment VALUES  
(1001, 1, 101, '2026-08-03'),  
(1002, 2, 102, '2026-08-04');
```

```
INSERT INTO Prescription VALUES  
(5001, 1001, 'Paracetamol'),  
(5002, 1002, 'Aspirin');
```

```
-- View data
```

```
SELECT * FROM Patient;  
SELECT * FROM Doctor;  
SELECT * FROM Appointment;  
SELECT * FROM Prescription;
```

```
-- Join query
```

```
SELECT  
    Patient.Name,  
    Doctor.DoctorName,  
    Appointment.Date,
```

Prescription.Medicine
FROM Prescription
JOIN Appointment
ON Prescription.AppointmentID = Appointment.AppointmentID
JOIN Patient
ON Appointment.PatientID = Patient.PatientID
JOIN Doctor
ON Appointment.DoctorID = Doctor.DoctorID;

4) Output

```
MySQL localhost:33060+ ssl SQL > -- Create database
Query OK, 0 rows affected (0.0010 sec)
MySQL localhost:33060+ ssl SQL >
MySQL localhost:33060+ ssl SQL > CREATE DATABASE hospital;
Query OK, 1 row affected (0.0206 sec)
MySQL localhost:33060+ ssl SQL >
MySQL localhost:33060+ ssl SQL > -- Use database
Query OK, 0 rows affected (0.0010 sec)
MySQL localhost:33060+ ssl SQL >
MySQL localhost:33060+ ssl SQL > USE hospital;
Default schema set to 'hospital'.
Fetching global names, object names from 'hospital' for auto-completion... Press ^C to stop.
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > -- Create Patient table
Query OK, 0 rows affected (0.0010 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > CREATE TABLE Patient (
    -> PatientID INT PRIMARY KEY,
    -> Name VARCHAR(50),
    -> Age INT
    -> );
Query OK, 0 rows affected (0.0600 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > -- Create Doctor table
Query OK, 0 rows affected (0.0006 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > CREATE TABLE Doctor (
    -> DoctorID INT PRIMARY KEY,
    -> DoctorName VARCHAR(50),
    -> Specialization VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.0527 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > -- Create Appointment table
Query OK, 0 rows affected (0.0010 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > CREATE TABLE Appointment (
    -> AppointmentID INT PRIMARY KEY,
    -> PatientID INT,
    -> DoctorID INT,
    -> Date DATE,
```

```

Query OK, 0 rows affected (0.1526 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > -- Create Prescription table
Query OK, 0 rows affected (0.0008 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > CREATE TABLE Prescription (
    -> PrescriptionID INT PRIMARY KEY,
    -> AppointmentID INT,
    -> Medicine VARCHAR(50),
    -> FOREIGN KEY (AppointmentID) REFERENCES Appointment(AppointmentID)
    -> );
Query OK, 0 rows affected (0.0752 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > -- Show tables
Query OK, 0 rows affected (0.0005 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > SHOW TABLES;
+-----+
| Tables_in_hospital |
+-----+
| appointment        |
| doctor             |
| patient            |
| prescription        |
+-----+
4 rows in set (0.0028 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > -- Display structure
Query OK, 0 rows affected (0.0006 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > DESCRIBE Patient;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| PatientID  | int       | NO   | PRI | NULL    |       |
| Name       | varchar(50) | YES  |     | NULL    |       |
| Age        | int       | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.0032 sec)

```

```

MySQL localhost:33060+ ssl hospital SQL > DESCRIBE Doctor;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| DoctorID   | int       | NO   | PRI | NULL    |       |
| DoctorName | varchar(50) | YES  |     | NULL    |       |
| Specialization | varchar(50) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.0029 sec)
MySQL localhost:33060+ ssl hospital SQL > DESCRIBE Appointment;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| AppointmentID | int       | NO   | PRI | NULL    |       |
| PatientID     | int       | YES  | MUL | NULL    |       |
| DoctorID      | int       | YES  | MUL | NULL    |       |
| Date          | date      | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.0026 sec)
MySQL localhost:33060+ ssl hospital SQL > DESCRIBE Prescription;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| PrescriptionID | int       | NO   | PRI | NULL    |       |
| AppointmentID  | int       | YES  | MUL | NULL    |       |
| Medicine       | varchar(50) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+

```

```

MySQL localhost:33060+ ssl hospital SQL > -- Insert sample values
Query OK, 0 rows affected (0.0005 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > INSERT INTO Patient VALUES
-> (1, 'Ram', 25),
-> (2, 'Priya', 30);

Query OK, 2 rows affected (0.0131 sec)

Records: 2 Duplicates: 0 Warnings: 0
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > INSERT INTO Doctor VALUES
-> (101, 'Dr. Kumar', 'Cardiology'),
-> (102, 'Dr. Meena', 'Neurology');

Query OK, 2 rows affected (0.0117 sec)

Records: 2 Duplicates: 0 Warnings: 0
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > INSERT INTO Appointment VALUES
-> (1001, 1, 101, '2026-08-03'),
-> (1002, 2, 102, '2026-08-04');

Query OK, 2 rows affected (0.0121 sec)

Records: 2 Duplicates: 0 Warnings: 0
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > INSERT INTO Prescription VALUES
-> (5001, 1001, 'Paracetamol'),
-> (5002, 1002, 'Aspirin');

Query OK, 2 rows affected (0.0141 sec)

Records: 2 Duplicates: 0 Warnings: 0
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > -- View data
Query OK, 0 rows affected (0.0007 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > SELECT * FROM Patient;
+-----+-----+-----+
| PatientID | Name | Age |
+-----+-----+-----+
|          1 | Ram  | 25  |
|          2 | Priya | 30  |
+-----+-----+-----+

```

```

2 rows in set (0.0015 sec)
MySQL localhost:33060+ ssl hospital SQL > SELECT * FROM Doctor;
+-----+-----+-----+
| DoctorID | DoctorName | Specialization |
+-----+-----+-----+
| 101 | Dr. Kumar | Cardiology |
| 102 | Dr. Meena | Neurology |
+-----+-----+-----+
2 rows in set (0.0013 sec)
MySQL localhost:33060+ ssl hospital SQL > SELECT * FROM Appointment;
+-----+-----+-----+-----+
| AppointmentID | PatientID | DoctorID | Date |
+-----+-----+-----+-----+
| 1001 | 1 | 101 | 2026-08-03 |
| 1002 | 2 | 102 | 2026-08-04 |
+-----+-----+-----+-----+
2 rows in set (0.0013 sec)
MySQL localhost:33060+ ssl hospital SQL > SELECT * FROM Prescription;
+-----+-----+-----+
| PrescriptionID | AppointmentID | Medicine |
+-----+-----+-----+
| 5001 | 1001 | Paracetamol |
| 5002 | 1002 | Aspirin |
+-----+-----+-----+
2 rows in set (0.0012 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > -- Join query
Query OK, 0 rows affected (0.0008 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > SELECT
-> Patient.Name,
-> Doctor.DoctorName,
-> Appointment.Date,
-> Prescription.Medicine
-> FROM Prescription
-> JOIN Appointment
-> ON Prescription.AppointmentID = Appointment.AppointmentID
-> JOIN Patient
-> ON Appointment.PatientID = Patient.PatientID
-> JOIN Doctor
-> ON Appointment.DoctorID = Doctor.DoctorID;

```

```

+-----+-----+-----+-----+
| Name | DoctorName | Date | Medicine |
+-----+-----+-----+-----+
| Ram | Dr. Kumar | 2026-08-03 | Paracetamol |
| Priya | Dr. Meena | 2026-08-04 | Aspirin |
+-----+-----+-----+-----+
2 rows in set (0.0018 sec)
MySQL localhost:33060+ ssl hospital SQL > |

```

5) Schema Analysis

Primary Keys

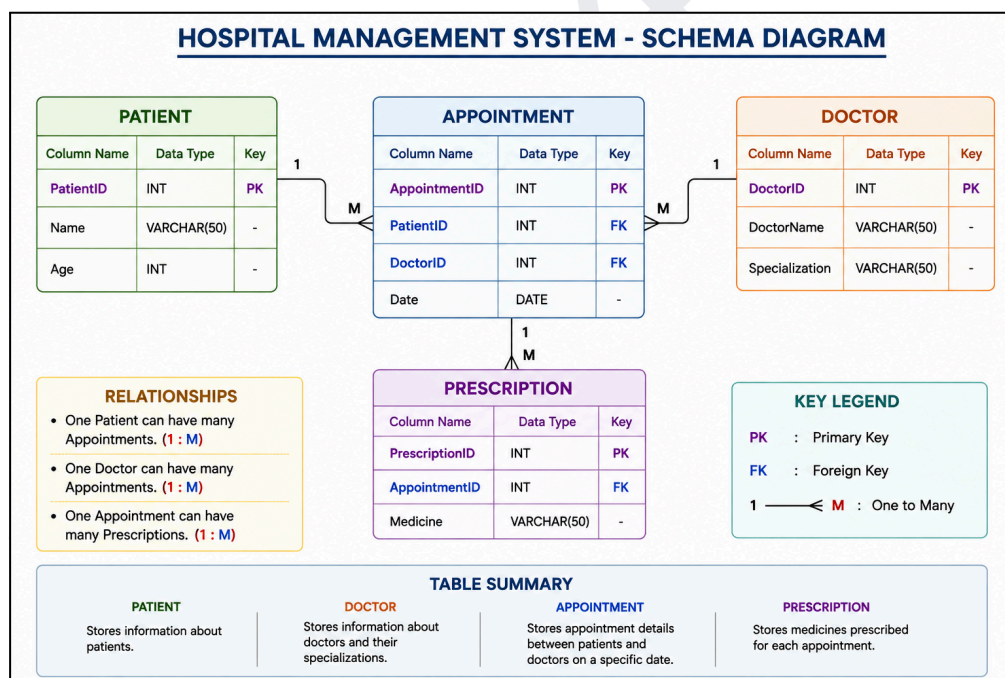
- PatientID
- DoctorID
- AppointmentID
- PrescriptionID

Foreign Keys

- Appointment.PatientID → Patient.PatientID
- Appointment.DoctorID → Doctor.DoctorID
- Prescription.AppointmentID → Appointment.AppointmentID

Normalization

- 1NF: Atomic values.
- 2NF: No partial dependency.
- 3NF: No transitive dependency.



6) Conclusion

The Hospital Management System schema was successfully created and analyzed using MySQL. The relationships between patients, doctors, appointments, and prescriptions were established using primary and foreign keys, ensuring data integrity and reducing redundancy.

3) Bus Ticket Reservation System

1) Aim

To create and analyze the Bus Ticket Reservation System schema using MySQL by defining tables, primary keys, foreign keys, and relationships.

2) Schema Design

Passenger Table

- PassengerID (PK)
- Name

Bus Table

- BusID (PK)
- Route

Booking Table

- BookingID (PK)
- PassengerID (FK)
- BusID (FK)
- TravelDate

Relationship Analysis

- One passenger can make many bookings (**1:M**).
- One bus can have many bookings (**1:M**).

3) MySQL Code

```
-- Create database
```

```
CREATE DATABASE bus_reservation;
```

```
-- Use database
```

```
USE bus_reservation;
```

```
-- Create Passenger table
```

```
CREATE TABLE Passenger (
```

```
    PassengerID INT PRIMARY KEY,
```

```
Name VARCHAR(50)
);
-- Create Bus table
CREATE TABLE Bus (
    BusID INT PRIMARY KEY,
    Route VARCHAR(100)
);
-- Create Booking table
CREATE TABLE Booking (
    BookingID INT PRIMARY KEY,
    PassengerID INT,
    BusID INT,
    TravelDate DATE,
    FOREIGN KEY (PassengerID) REFERENCES Passenger(PassengerID),
    FOREIGN KEY (BusID) REFERENCES Bus(BusID)
);
-- Display tables
SHOW TABLES;
-- Display structure
DESCRIBE Passenger;
DESCRIBE Bus;
DESCRIBE Booking;
-- Insert sample data
INSERT INTO Passenger VALUES
(1, 'Ram'),
(2, 'Priya');
INSERT INTO Bus VALUES
```

(101, 'Chennai to Coimbatore'),

(102, 'Madurai to Trichy');

INSERT INTO Booking VALUES

(1001, 1, 101, '2026-08-03'),

(1002, 2, 102, '2026-08-04');

-- View data

SELECT * FROM Passenger;

SELECT * FROM Bus;

SELECT * FROM Booking;

-- Join query

SELECT

 Passenger.Name,

 Bus.Route,

 Booking.TravelDate

FROM Booking

JOIN Passenger

ON Booking.PassengerID = Passenger.PassengerID

JOIN Bus

ON Booking.BusID = Bus.BusID;

4) Output

```
2 rows in set (0.0018 sec)
MySQL localhost:33060+ ssl hospital SQL > -- Create database
Query OK, 0 rows affected (0.0016 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > CREATE DATABASE bus_reservation;
Query OK, 1 row affected (0.0283 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > -- Use database
Query OK, 0 rows affected (0.0007 sec)
MySQL localhost:33060+ ssl hospital SQL >
MySQL localhost:33060+ ssl hospital SQL > USE bus_reservation;
Default schema set to 'bus_reservation'.
Fetching global names, object names from 'bus_reservation' for auto-completion... Press ^C to stop.
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > -- Create Passenger table
Query OK, 0 rows affected (0.0010 sec)
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > CREATE TABLE Passenger (
-> PassengerID INT PRIMARY KEY,
-> Name VARCHAR(50)
-> );
Query OK, 0 rows affected (0.0951 sec)
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > -- Create Bus table
Query OK, 0 rows affected (0.0008 sec)
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > CREATE TABLE Bus (
-> BusID INT PRIMARY KEY,
-> Route VARCHAR(100)
-> );
Query OK, 0 rows affected (0.0731 sec)
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > -- Create Booking table
Query OK, 0 rows affected (0.0008 sec)
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > CREATE TABLE Booking (
-> BookingID INT PRIMARY KEY,
-> PassengerID INT,
-> BusID INT,
-> TravelDate DATE,
-> FOREIGN KEY (PassengerID) REFERENCES Passenger(PassengerID),
```

```

MySQL localhost:33060+ ssl bus_reservation SQL > -- Display tables
Query OK, 0 rows affected (0.0012 sec)
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > SHOW TABLES;
+-----+
| Tables_in_bus_reservation |
+-----+
| booking                    |
| bus                        |
| passenger                  |
+-----+
3 rows in set (0.0043 sec)
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > -- Display structure
Query OK, 0 rows affected (0.0009 sec)
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > DESCRIBE Passenger;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| PassengerID | int       | NO   | PRI | NULL    |      |
| Name       | varchar(50) | YES  |     | NULL    |      |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.0032 sec)
MySQL localhost:33060+ ssl bus_reservation SQL > DESCRIBE Bus;
+-----+-----+-----+-----+-----+-----+
| Field | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| BusID | int       | NO   | PRI | NULL    |      |
| Route | varchar(100) | YES  |     | NULL    |      |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.0028 sec)
MySQL localhost:33060+ ssl bus_reservation SQL > DESCRIBE Booking;
+-----+-----+-----+-----+-----+-----+
| Field      | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| BookingID  | int  | NO   | PRI | NULL    |      |
| PassengerID | int  | YES  | MUL | NULL    |      |
| BusID      | int  | YES  | MUL | NULL    |      |
| TravelDate | date | YES  |     | NULL    |      |
+-----+-----+-----+-----+-----+-----+

```

```

MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > INSERT INTO Passenger VALUES
-> (1, 'Ram'),
-> (2, 'Priya');

Query OK, 2 rows affected (0.0164 sec)

Records: 2 Duplicates: 0 Warnings: 0
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > INSERT INTO Bus VALUES
-> (101, 'Chennai to Coimbatore'),
-> (102, 'Madurai to Trichy');

Query OK, 2 rows affected (0.0093 sec)

Records: 2 Duplicates: 0 Warnings: 0
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > INSERT INTO Booking VALUES
-> (1001, 1, 101, '2026-08-03'),
-> (1002, 2, 102, '2026-08-04');

Query OK, 2 rows affected (0.0136 sec)

Records: 2 Duplicates: 0 Warnings: 0
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > -- View data
Query OK, 0 rows affected (0.0009 sec)
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > SELECT * FROM Passenger;
+-----+-----+
| PassengerID | Name |
+-----+-----+
|          1 | Ram  |
|          2 | Priya|
+-----+-----+
2 rows in set (0.0010 sec)
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > SELECT * FROM Bus;
+-----+-----+
| BusID | Route |
+-----+-----+
|    101 | Chennai to Coimbatore |
|    102 | Madurai to Trichy    |
+-----+-----+

```

```

2 rows in set (0.0011 sec)
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > SELECT * FROM Booking;
+-----+-----+-----+-----+
| BookingID | PassengerID | BusID | TravelDate |
+-----+-----+-----+-----+
|        1001 |          1 |    101 | 2026-08-03 |
|        1002 |          2 |    102 | 2026-08-04 |
+-----+-----+-----+-----+
2 rows in set (0.0009 sec)
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > -- Join query
Query OK, 0 rows affected (0.0006 sec)
MySQL localhost:33060+ ssl bus_reservation SQL >
MySQL localhost:33060+ ssl bus_reservation SQL > SELECT
->     Passenger.Name,
->     Bus.Route,
->     Booking.TravelDate
-> FROM Booking
-> JOIN Passenger
-> ON Booking.PassengerID = Passenger.PassengerID
-> JOIN Bus
-> ON Booking.BusID = Bus.BusID;
+-----+-----+-----+
| Name | Route | TravelDate |
+-----+-----+-----+
| Ram  | Chennai to Coimbatore | 2026-08-03 |
| Priya | Madurai to Trichy    | 2026-08-04 |
+-----+-----+-----+
2 rows in set (0.0016 sec)
MySQL localhost:33060+ ssl bus_reservation SQL >

```

5) Schema Analysis

Primary Keys

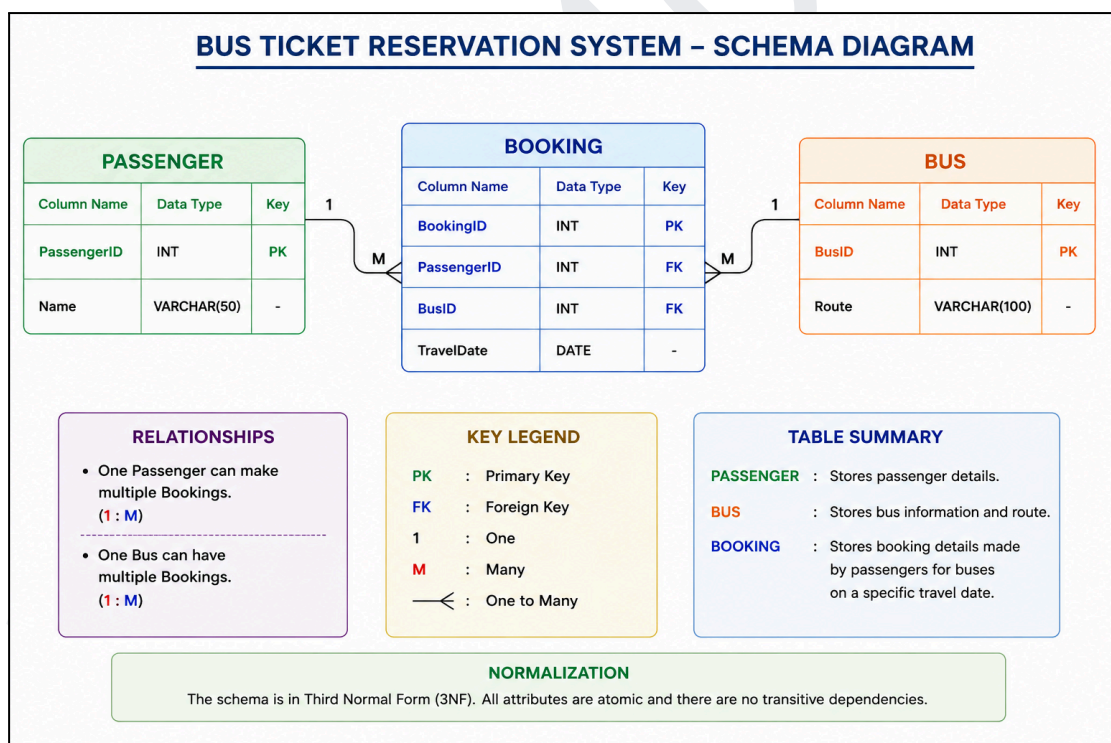
- PassengerID
- BusID
- BookingID

Foreign Keys

- Booking.PassengerID → Passenger.PassengerID
- Booking.BusID → Bus.BusID

Normalization

- 1NF: Atomic values.
- 2NF: No partial dependency.
- 3NF: No transitive dependency.



6) Conclusion

The Bus Ticket Reservation System schema was successfully created and analyzed using MySQL. The relationships between passengers, buses, and bookings were established using primary and foreign keys, ensuring data integrity and reducing redundancy.